Documentação técnica - AgendaFront

Gerado em: 16/05/2026

1. Visão geral

O AgendaFront é o frontend da Agenda de Contatos da GMMS Tech Solutions. Ele foi desenvolvido como uma aplicação web em Vue.js para autenticar usuários e permitir a gestão de contatos por meio de operações de cadastro, consulta, edição, exclusão, busca e paginação.

A aplicação consome a API CoreFlow pelos endpoints `/api/Auth` e `/api/User`. O frontend não guarda os contatos como fonte definitiva de dados; ele atua como camada de interface, validação inicial, sessão do usuário e comunicação com o backend.

2. Tipo de aplicação

O projeto é uma SPA, ou seja, uma Single Page Application. A evidência principal está em `src/main.js`, `src/App.vue`, `src/router/index.js` e no fallback do Nginx em `nginx/default.conf.template`, que redireciona as rotas do navegador para `index.html` usando `try_files $uri $uri/ /index.html`.

Classificação do projeto:

```
Tipo | Se aplica? | Observação
SPA | Sim | A navegação ocorre no cliente com Vue Router, sem recarregar páginas HTML diferentes.
MPA | Não | Não há múltiplas páginas HTML renderizadas pelo servidor; existe um `index.html` principal.
SSR | Não | Não há renderização no servidor. O build gera arquivos estáticos servidos pelo Vite ou Nginx.
PWA | Não identificada | Não há service worker, manifest web app ou estratégia offline no código atual.
RPA | Não | RPA é automação robótica de processos. Este projeto é uma interface web, não um robô de automação.
ERP | Não como produto completo | O projeto é um módulo/tela de agenda e usuários. Ele poderia integrar um ERP, mas não contém módulos típicos de ERP como financeiro, estoque, vendas e fiscal.
```

Portanto, a definição mais correta é: SPA frontend de agenda de contatos, com autenticação JWT, CRUD de contatos e integração com uma API backend.

3. Arquitetura utilizada

A arquitetura é uma SPA client-side com separação simples por responsabilidade. O projeto não usa uma arquitetura formal como Clean Architecture completa no frontend, mas segue uma organização em camadas práticas:

```
Camada | Função | Onde está
Bootstrap da aplicação | Cria a aplicação Vue, registra Pinia e Vue Router e monta no elemento `#app`. | `src/main.js`
Composição raiz | Renderiza a rota ativa. | `src/App.vue`
Roteamento | Define as rotas `/login` e `/agenda`, redireciona `/` para `/agenda` e aplica guarda de autenticação. | `src/router/index.js`
Views/telas | Contêm a interface, eventos de usuário e integração com stores. | `src/views/LoginView.vue`, `src/views/ContactsView.vue`
Estado global | Centraliza estado, getters e ações de autenticação e contatos. | `src/stores/authStore.js`, `src/stores/contactsStore.js`
Serviços de API | Encapsulam chamadas HTTP e montagem de payloads. | `src/services/apiClient.js`,
```

`src/services/authApi.js`, `src/services/contactsApi.js`
Sessão local | Normaliza, salva, lê e remove a sessão JWT do `localStorage`. | `src/services/authSession.js`
Regras auxiliares | Calculam força de senha e validações/formatações de contato. | `src/services/passwordStrength.js`, `src/stores/contactsStore.js`
Estilos | Define o visual global com Tailwind CSS e classes customizadas. | `src/styles.css`
Testes | Validam tela de login, tela de contatos e regras da store de contatos. | `src/views/__tests__`, `src/stores/__tests__`
Build e deploy | Configura Vite, Docker e Nginx para servir a SPA e encaminhar chamadas `/api`. | `vite.config.js`, `Dockerfile`, `docker-compose.yml`, `nginx/default.conf.template`

O fluxo arquitetural principal é:

1. O usuário acessa a SPA.
2. O Vue Router decide qual tela será exibida.
3. Rotas protegidas chamam a store de autenticação para validar a sessão.
4. As views acionam stores do Pinia.
5. As stores executam validações e chamam serviços de API.
6. Os serviços usam `apiClient` para enviar requisições HTTP ao backend.
7. A resposta atualiza o estado da store.
8. O Vue re-renderiza a interface de forma reativa.

4. Tecnologias e onde são usadas

Vue 3

Vue é o framework principal da interface. O projeto usa Single File Components (`.vue`) e Composition API com `<script setup>`.

Uso no projeto:

Arquivo | Uso
`src/main.js` | Criação e montagem da aplicação Vue.
`src/App.vue` | Renderização do componente da rota ativa com `<RouterView />`.
`src/views/LoginView.vue` | Tela de autenticação.
`src/views/ContactsView.vue` | Tela principal da agenda de contatos.

Vite

Vite é usado como ferramenta de desenvolvimento, build e proxy local.

Uso no projeto:

Arquivo | Uso
`package.json` | Scripts `npm run dev`, `npm run build`, `npm run preview` e `npm run test:unit`.
`vite.config.js` | Plugins Vue e Tailwind, ambiente de teste jsdom, porta do frontend e proxy `/api`.
`index.html` | Entrada HTML usada pelo Vite para carregar a SPA.

O proxy do Vite encaminha chamadas feitas para `/api` ao backend CoreFlow configurado em variáveis `VITE_CORE_FLOW_*`.

Vue Router

Vue Router controla a navegação client-side.

Uso no projeto:

```
Arquivo | Uso
`src/router/index.js` | Define `/login`, `/agenda`, redirecionamento `/` e guarda de rota autenticada.
```

Regras de rota:

* `/` redireciona para `/agenda`.
* `/login` exibe a tela de login.
* `/agenda` exige autenticação.
* Se a sessão não for válida, o usuário é enviado para `/login`.
* Se o usuário já estiver autenticado e acessar `/login`, ele é redirecionado para `/agenda`.

Pinia

Pinia é usado para estado global.

Uso no projeto:

```
Arquivo | Uso
`src/stores/index.js` | Cria a instância do Pinia.
`src/stores/authStore.js` | Estado e ações de login, autenticação, troca de senha e logout.
`src/stores/contactsStore.js` | Estado, validações, filtros, paginação e CRUD de contatos.
```

Tailwind CSS

Tailwind é usado para compor estilos com utilitários e `@apply`.

Uso no projeto:

```
Arquivo | Uso
`src/styles.css` | Importa Tailwind e define classes globais como `.auth-page`, `.agenda-page`, `.panel`,
`.primary-action`, `.contact-form`, `.contact-list` e responsividade.
`vite.config.js` | Registra o plugin `@tailwindcss/vite`.
```

Lucide Vue

Lucide Vue fornece ícones para botões e estados visuais.

Uso no projeto:

```
Arquivo | Uso
`src/views/LoginView.vue` | Ícones de login, loading e mostrar/ocultar senha.
`src/views/ContactsView.vue` | Ícones de cadastro, salvar, editar, excluir, consultar, buscar, atualizar,
paginação, logout e senha.
```

Fetch API e camada de serviços

As chamadas HTTP são centralizadas em serviços para evitar espalhar lógica de API dentro das views.

Uso no projeto:

```
Arquivo | Uso
`src/services/apiClient.js` | Define `apiRequest`, base URL, headers, tradução de mensagens de erro e
tratamento de sessão expirada.
`src/services/authApi.js` | Consome login, validação de token e troca de senha.
```

`src/services/contactsApi.js` | Consome listagem, consulta por id, cadastro, edição e exclusão de contatos.

localStorage

O `localStorage` é usado para manter informações locais entre recarregamentos.

Uso no projeto:

Chave | Finalidade | Arquivo
`agenda-front-auth` | Guarda sessão JWT, tipo do token, expiração e usuário autenticado. | `src/services/authSession.js`
`agenda-front-recent-contact-ids` | Guarda IDs de contatos recém-cadastrados para priorização visual quando a API não retorna data de criação. | `src/stores/contactsStore.js`

Vitest, Vue Test Utils e jsdom

São usados para testes unitários e de componentes.

Uso no projeto:

Arquivo | Cobertura
`src/views/__tests__/LoginView.spec.js` | Login, erro de credenciais, persistência de sessão e alternância de visibilidade de senha.
`src/views/__tests__/ContactsView.spec.js` | Lista de contatos, paginação, cadastro, edição, validações, troca de senha e exibição do usuário logado.
`src/stores/__tests__/contactsStore.spec.js` | Validação de e-mail, telefone, senha padrão e ordenação por contatos recentes.

Docker e Nginx

Docker empacota o frontend e Nginx serve os arquivos estáticos em produção.

Uso no projeto:

Arquivo | Uso
`Dockerfile` | Build com Node 24 Alpine e runtime com Nginx 1.29 Alpine.
`docker-compose.yml` | Sobe o container do frontend.
`nginx/default.conf.template` | Serve a SPA e encaminha `/api/` para `API_PROXY_TARGET`.

5. Integração com backend

O frontend usa `VITE_API_BASE_URL=/api` por padrão. Isso permite que o browser sempre chame uma rota relativa `/api`, enquanto Vite ou Nginx fazem o proxy para a API CoreFlow.

Endpoints consumidos:

Operação | Método e endpoint | Arquivo consumidor
Login | `POST /api/Auth/login` | `src/services/authApi.js`
Validar sessão | `GET /api/Auth/authenticate` | `src/services/authApi.js`, `src/stores/authStore.js`
Listar contatos | `GET /api/User` | `src/services/contactsApi.js`
Consultar contato | `GET /api/User/{id}` | `src/services/contactsApi.js`
Cadastrar contato | `POST /api/User` | `src/services/contactsApi.js`, `src/stores/contactsStore.js`
Editar contato | `PUT /api/User/{id}` | `src/services/contactsApi.js`, `src/stores/contactsStore.js`
Alterar senha própria | `PATCH /api/User/me/password` | `src/services/authApi.js`, `src/stores/authStore.js`
Excluir contato | `DELETE /api/User/{id}` | `src/services/contactsApi.js`

Todas as chamadas protegidas usam `Authorization: Bearer <token>`, montado a partir da sessão salva no `localStorage`.

6. Regra de negócio do cadastro de contatos

O cadastro de contatos acontece na tela `src/views/ContactsView.vue`, mas as regras principais estão em `src/stores/contactsStore.js` e a chamada HTTP final está em `src/services/contactsApi.js`.

6.1 Pré-condição

O usuário precisa estar autenticado para acessar `/agenda`. A rota tem `meta: { requiresAuth: true }` e o guard em `src/router/index.js` chama `authStore.authenticate()`. Sem sessão válida, o usuário é redirecionado para `/login`.

6.2 Campos exibidos no cadastro

O formulário de novo contato exibe apenas:

```
Campo | Regra
Nome | Obrigatório, máximo de 50 caracteres.
E-mail | Obrigatório, máximo de 40 caracteres e formato válido.
Telefone | Obrigatório, celular brasileiro com 11 dígitos locais: DDD + número.
```

A senha não aparece no formulário de cadastro de contato.

6.3 Validação de nome

Regra implementada em `validateNameField()`:

* o nome é obrigatório;
* o nome é validado após `trim()`;
* o limite máximo é `NAME_MAX_LENGTH = 50`;
* se inválido, o cadastro é bloqueado e a mensagem de campo é exibida.

6.4 Validação de e-mail

Regra implementada em `validateEmailField()`:

* o e-mail é obrigatório;
* o limite máximo é `EMAIL_MAX_LENGTH = 40`;
* o formato precisa passar no padrão `EMAIL_PATTERN`;
* erros retornados pela API, como e-mail duplicado ou inválido, são traduzidos em `src/services/apiClient.js`.

6.5 Validação e normalização de telefone

Regra implementada em `setPhone()`, `formatBrazilianMobilePhone()`, `isValidBrazilianMobilePhone()` e `toBrazilianE164Phone()`:

* o telefone é obrigatório;
* o frontend aceita digitação numérica, telefone com máscara ou telefone colado com `+55`;
* a interface mostra o telefone como `+55 (xx) xxxxx-xxxx`;

* o número precisa conter 11 dígitos locais, sendo 2 de DDD e 9 do celular;
* no payload enviado ao backend, o telefone é convertido para formato compacto com DDI: `+55DDDNÚMERO`, por exemplo `+5581997236704`;
* erros retornados pela API, como telefone duplicado ou inválido, são traduzidos em `src/services/apiClient.js`.

6.6 Payload de cadastro

Ao salvar um novo contato, `saveContact()` monta o payload base:

```
{
"id": "",
"name": "Nome do contato",
"email": "email@dominio.com",
"phone": "+5581997236704"
}
```

Como o backend usa o endpoint `/User`, o frontend acrescenta automaticamente uma senha padrão ao cadastrar:

```
{
"name": "Nome do contato",
"email": "email@dominio.com",
"phone": "+5581997236704",
"password": "Admin@123456"
}
```

A senha padrão está declarada como `DEFAULT_CONTACT_PASSWORD = 'Admin@123456'` em `src/stores/contactsStore.js` e é coberta por teste em `src/stores/__tests__/contactsStore.spec.js`.

6.7 Envio para a API

O cadastro chama `contactsApi.create()` em `src/services/contactsApi.js`, que envia:

* método: `POST`;
* endpoint: `/api/User`;
* autenticação: `Authorization: Bearer <token>`;
* corpo: `name`, `email`, `phone` e `password`.

Antes do envio, `contactsApi` aplica `trim()` em nome, e-mail e telefone.

6.8 Comportamento após cadastro com sucesso

Quando a API confirma o cadastro:

* o frontend mostra a mensagem `Contato cadastrado.`;
* o formulário é limpo;
* a lista de contatos é recarregada;
* se o contato recém-criado for encontrado por e-mail e telefone, seu ID é promovido para a lista local de recentes;
* a página atual volta para a primeira página;
* a listagem prioriza contatos por data de criação quando a API retorna um campo como `createdAt`; se não houver data, usa a lista local de contatos recentes.

6.9 Diferença entre cadastro e edição

No cadastro, o frontend envia uma senha padrão porque o endpoint usado é de usuário:

* `POST /api/User`
* campos: `name`, `email`, `phone`, `password`

Na edição, o frontend não altera senha:

* `PUT /api/User/{id}`
* campos: `id`, `name`, `email`, `phone`

Alteração de senha é uma regra separada, feita apenas para o usuário autenticado em:

* `PATCH /api/User/me/password`

6.10 Regras de consulta, busca, paginação e exclusão

Além do cadastro, a tela de agenda possui:

```
Funcionalidade | Regra
Consulta | Botão de visualizar chama `GET /api/User/{id}` e mostra detalhes do contato selecionado.
Busca | Filtra localmente por nome, e-mail ou telefone.
Paginação | Exibe 10 contatos por página, definido por `CONTACTS_PER_PAGE = 10`.
Edição | Preenche o formulário com o contato selecionado e salva por `PUT /api/User/{id}`.
Exclusão | Pede confirmação com `window.confirm()` e remove por `DELETE /api/User/{id}`.
```

7. Segurança e sessão

A autenticação usa JWT:

* o login chama `POST /api/Auth/login`;
* a resposta é normalizada por `normalizeAuthResponse()`;
* a sessão é salva no `localStorage` com token, tipo, expiração, usuário e data de autenticação;
* a validade considera a expiração do token com uma margem de 30 segundos;
* rotas protegidas validam a sessão antes de permitir acesso;
* chamadas protegidas recebem header `Authorization`;
* ao receber erro 401 em chamada autenticada, a sessão local é removida.

Observação técnica: por usar `localStorage`, a sessão persiste após recarregar a página, mas também exige cuidado contra riscos de XSS. A segurança final depende de o frontend manter a aplicação livre de injeção de scripts e o backend validar todas as regras críticas.

8. Testes existentes

O projeto possui testes unitários e de componente com Vitest:

* `npm run test:unit` executa a suíte;
* `contactsStore.spec.js` cobre validações de e-mail, telefone, senha padrão e ordenação;
* `ContactsView.spec.js` cobre cadastro, edição, paginação, validações e troca de senha;
* `LoginView.spec.js` cobre autenticação, erro de login, persistência da sessão e visibilidade da

senha.

Esses testes ajudam a garantir que as principais regras de negócio do frontend continuem funcionando após alterações.

9. Resumo executivo

O AgendaFront é uma SPA Vue 3 com arquitetura modular por responsabilidade. Ela utiliza Vue Router para navegação, Pinia para estado global, serviços JavaScript para integração com API, Tailwind para interface, Vitest para testes e Docker/Nginx para distribuição.

O projeto não é RPA e não é um ERP completo. Ele é uma aplicação frontend de agenda de contatos, com autenticação JWT e CRUD sobre usuários/contatos da API CoreFlow.

A principal regra de negócio do cadastro é: um usuário autenticado cadastra contatos informando nome, e-mail e celular brasileiro; o frontend valida esses campos, formata o telefone, envia o telefone normalizado e adiciona internamente a senha padrão `Admin@123456` porque o backend cadastra o contato pelo endpoint de usuário.